

# MERN Stack Coding Assessment

Stack: MongoDB · Express · React · Node.js

## Problem Statement: "TaskFlow" — A Smart Task Manager with Priority Scoring

Build a mini task management application where users can create tasks, and the backend automatically calculates a **priority score** for each task based on its deadline and importance. The frontend should display tasks sorted by this score, with the ability to filter and mark tasks as complete.

### Objective

Evaluate the candidate's ability to:

- 1. Design and implement RESTful APIs with Express.js and MongoDB (Mongoose).
- 2. Apply server-side business logic (computed/derived fields).
- 3. Build a responsive React UI with state management (hooks) and asynchronous data fetching.
- 4. Handle edge cases, validation, and error states across the stack.
- 5. Write clean, modular, and readable code under time pressure.

## Part 1 — Backend (Node.js + Express + MongoDB)

Data Model: **Task**

| Field                      | Type     | Constraints                                                                              |
|----------------------------|----------|------------------------------------------------------------------------------------------|
| <code>_id</code>           | ObjectId | Auto-generated                                                                           |
| <code>title</code>         | String   | Required, 3-100 chars                                                                    |
| <code>description</code>   | String   | Optional, max 500 chars                                                                  |
| <code>importance</code>    | Number   | Required, integer 1-5 (1 = low, 5 = critical)                                            |
| <code>dueDate</code>       | Date     | Required, must be a future date on creation                                              |
| <code>status</code>        | String   | Enum: <code>"pending"</code>   <code>"completed"</code> , default <code>"pending"</code> |
| <code>createdAt</code>     | Date     | Auto-generated                                                                           |
| <code>priorityScore</code> | Number   | <b>Computed</b> (see formula below) — returned in API responses                          |

### Priority Score Formula

For each task, compute:

```
priorityScore = (importance * 10) + (100 / max(daysUntilDue, 1))
```

Where `daysUntilDue` is the number of full days between `now` and `dueDate` (rounded down, minimum 1 to avoid division by zero). For completed tasks, `priorityScore` should always be `0`.

The score must be computed **on the server** at read time (do not store it). Return it in every task response, rounded to 2 decimals.

### API Specs

Base path: `/bfhl/tasks` — all routes **must** be served under the `/bfhl` prefix.

| # | Method | Endpoint        | Description                                                                        |
|---|--------|-----------------|------------------------------------------------------------------------------------|
| 1 | POST   | /bfhl/tasks     | Create a new task. Validate all fields. Return the created task with score.        |
| 2 | GET    | /bfhl/tasks     | List all tasks, sorted by <code>priorityScore</code> DESC. Supports query filters. |
| 3 | PATCH  | /bfhl/tasks/:id | Update a task (any subset of editable fields, including <code>status</code> ).     |
| 4 | DELETE | /bfhl/tasks/:id | Delete a task by ID.                                                               |

#### Query Filters for `GET /bfhl/tasks`

- `?status=pending` or `?status=completed` — filter by status
- `?minImportance=3` — only tasks with `importance >= 3`

Both filters should be combinable (e.g. `?status=pending&minImportance=3`).

#### Validation & Error Handling

- Return `400` with a clear error message for invalid inputs (missing fields, bad types, past `dueDate` on create, importance out of range).
- Return `404` when a task ID does not exist.
- Return `500` only for unexpected server errors.

#### Sample Response ( `GET /bfhl/tasks` )

```
[
  {
    "_id": "65f...",
    "title": "Submit project report",
    "description": "Final draft for review",
    "importance": 5,
    "dueDate": "2026-05-25T00:00:00.000Z",
    "status": "pending",
    "createdAt": "2026-05-20T10:00:00.000Z",
    "priorityScore": 70.00
  }
]
```

## Part 2 — Frontend (React)

Build a single-page interface that consumes the API above.

### Frontend Specs

#### 1. Task List View

- Fetch and display all tasks on mount, sorted by `priorityScore` (already sorted by the API).
- Each task card shows: title, description, importance (as stars or a badge), due date (human-readable, e.g. "in 3 days"), status, and priority score.
- Highlight tasks with `priorityScore >= 50` visually (e.g. red border or "High Priority" badge).

#### 2. Create Task Form

- Fields: title, description, importance (dropdown or slider 1–5), dueDate (date picker).
- Client-side validation matching the backend rules. Disable submit while a request is in flight.
- On success, the new task appears in the list without a full page reload.

#### 3. Filters

- A toggle/dropdown for status ( `All` / `Pending` / `Completed` ).
- A numeric input or slider for minimum importance.
- Filters update the list by re-querying the API.

#### 4. Actions per Task

- "Mark as Complete" button (calls PATCH; should re-fetch or update local state).
- "Delete" button with a confirm step.

## 5. UX Requirements

- Show a loading state while fetching.
- Show an empty state when no tasks match the filters.
- Show inline error messages for failed requests.

Styling: minimal CSS is fine — no UI library required. Functional clarity matters more than visual polish.

## Deliverables

- A `backend/` folder with the Express + MongoDB code.
- A `frontend/` folder with the React app.
- A public **GitHub repository** containing both folders (a monorepo, or two repos linked from a parent — either is fine).
- The project must be **deployed and publicly accessible**. Suggested free-tier providers:
  - **Frontend (React)**: Vercel, Netlify, or Render Static Site
  - **Backend (Express + MongoDB)**: Render, Railway, or Fly.io (use MongoDB Atlas free tier for the database)
- Submit two links: the **GitHub repo URL** and the **live deployed frontend URL**. The frontend must successfully talk to the deployed backend.
- A `README.md` is *optional*, but if included should cover setup steps and any assumptions made.

### Bonus Question (Optional — attempt only if time permits)

Add a `GET /bfhl/tasks/stats` endpoint that returns aggregate analytics using the **MongoDB aggregation pipeline** (not in-memory JavaScript):

```
{
  "totalTasks": 12,
  "pendingTasks": 8,
  "completedTasks": 4,
  "averageImportance": 3.25,
  "overdueTasks": 2,
  "tasksByImportance": {
    "1": 1, "2": 2, "3": 3, "4": 4, "5": 2
  }
}
```

- `overdueTasks` = pending tasks where `dueDate < now`.
- The implementation **must** use MongoDB's aggregation framework (`$group`, `$match`, `$cond`, `$facet`, etc.) — solutions that fetch all tasks and compute in Node.js will not be awarded bonus points.
- On the frontend, display these stats in a small dashboard card at the top of the page.

## Evaluation Notes

Your submission must meet the following concrete criteria. Reviewers will test against these directly.

- Your API must respond under **3 seconds** for every endpoint, including the deployed environment (cold-start hits excluded — keep the service warm if your provider sleeps).
- The deployed backend must return HTTP `200` on `GET /bfhl/tasks` when the database is empty (returning an empty array `[]`), not `404` or `500`.
- The `priorityScore` field must be present in every task response and must match the formula exactly, rounded to **2 decimals**.
- Tasks in `GET /bfhl/tasks` must be sorted by `priorityScore` in **descending** order.
- Invalid input (missing required fields, importance outside 1-5, past `dueDate`, malformed ID) must return HTTP `400` with a JSON body containing an `error` message — not a generic `500` or an HTML error page.
- The deployed frontend URL must successfully call the deployed backend URL — no `localhost` references, no CORS errors, no mixed-content (HTTP/HTTPS) issues in the browser console.
- The frontend must visibly handle three states: **loading** (while fetching), **empty** (when no tasks match filters), and **error** (when an API call fails).

- Creating a task from the frontend must reflect in the list **without a full page reload**.
- Filters ( `status` , `minImportance` ) must work both independently and combined — verified by hitting the deployed API directly with query params.
- Tasks with `priorityScore >= 50` must be visually distinguishable in the UI (border, badge, color — any clear indicator).
- **Bonus:** the `/bfhl/tasks/stats` endpoint, if attempted, must use MongoDB's aggregation framework ( `$group` , `$facet` , `$cond` , etc.). Reviewers will inspect the code — solutions that fetch all tasks and compute totals in JavaScript will not receive bonus credit, even if the numbers are correct.

**Plagiarism check:** All submitted GitHub repositories will be run through automated plagiarism and code-similarity detection tools. Submissions found to be copied from public repositories, other candidates, or AI-generated boilerplate without meaningful modification will be **disqualified**. You may reference documentation and standard tutorials, but the code you submit must be your own work.

## Submission Notes for Candidates

---

- You may use any npm packages (axios, mongoose, cors, dotenv, etc.).
- Use a local MongoDB instance or MongoDB Atlas free tier.
- Commit progress frequently; partial working solutions score better than incomplete "perfect" attempts.
- Focus on correctness first, then polish.

**Good luck!**